

Smart Lender: Applicant Credibility Prediction for Loan Approval

Date	06 July 2026
Team ID	TMID-SL-2026-01
Project Name	SmartLender - Applicant Credibility Prediction for Loan Approval
Maximum Marks	—

Project Description

Smart Lender is a machine-learning-powered web application that predicts the creditworthiness of loan applicants, helping banks and financial institutions make faster, data-driven loan approval decisions. The platform leverages four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost — to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model — XGBoost, reaching 96.5% training accuracy and 81.0% testing accuracy in this build — is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Lender gives applicants and credit officers a seamless interface to submit details and instantly receive an approval prediction, reducing non-performing assets and improving the credit evaluation process.

Scenarios

Scenario 1 — Fast-track approval for low-risk applicants: A credit officer enters details for a salaried applicant with a good credit history and stable income. The model predicts approval with high confidence, letting the officer fast-track the application without a full manual review.

Scenario 2 — High-risk applicant detection: An applicant with irregular self-employment income and no recorded credit history submits a loan request. The system flags the application as high-risk, prompting further scrutiny or document verification before proceeding.

Scenario 3 — Batch evaluation during high volume: A financial analyst uses the platform to screen multiple applicants during a high-volume period, relying on the XGBoost model's predictions to triage which applications need closest attention first.

Technical Architecture

Smart Lender follows a layered architecture: a Presentation layer (home.html, predict.html, submit.html) collects applicant details and shows results; a Flask Application layer (app.py) routes requests, validates input, and calls the Prediction Engine; the Prediction Engine loads the serialized model (rdf.pkl) and scaler (scale1.pkl) to convert user input into a scored feature vector; and the Machine Learning Pipeline (offline) handles dataset collection, preprocessing, balancing, scaling, training, evaluation, and model storage.

Entity Relationship Diagram

The system's data model centers on USER, APPLICANT_PROFILE, CREDIT_HISTORY, LOAN_APPLICATION, MODEL, and PREDICTION_RESULT entities. A USER (credit officer / applicant) can submit multiple LOAN_APPLICATIONS; each APPLICANT_PROFILE links to loan requests and to one CREDIT_HISTORY record; every LOAN_APPLICATION produces one PREDICTION_RESULT, generated by a specific MODEL (e.g., the deployed XGBoost model). This structure keeps applicant data, credit history, and model outputs traceable back to the originating loan request.

Pre-requisites

- Python programming proficiency
- Flask framework knowledge (routing, templates, form handling)
- scikit-learn / XGBoost familiarity (classification, evaluation metrics)
- Pandas / NumPy for data manipulation
- HTML & CSS for the Flask templates
- Git & GitHub for version control

Project Workflow

Milestone 1: Data Collection & Architecture Design

- Activity 1.1: Download the loan eligibility dataset and store it under Dataset/.
- Activity 1.2: Define the application architecture — data flow, ML workflow, and Flask deployment structure — captured in the Solution Architecture document.

Milestone 2: Visualizing and Analyzing the Data

- Activity 2.1: Import and read the dataset with Pandas.
- Activity 2.2: Perform univariate analysis (count plots for Gender/Education, distribution plots for ApplicantIncome/Credit_History).
- Activity 2.3: Perform bivariate analysis (Married x Gender, Self_Employed x Education, Property_Area x Loan_Amount_Term).
- Activity 2.4: Perform multivariate analysis (swarm plot of Gender x ApplicantIncome x Loan_Status).

Milestone 3: Data Pre-Processing

- Activity 3.1: Identify and fill missing values (mode for categorical, mean for numeric).
- Activity 3.2: Encode categorical features (Gender, Married, Education, Self_Employed, Property_Area, Loan_Status) to numeric codes; clean the '3+' Dependents value.
- Activity 3.3: Balance the training classes with SMOTE.
- Activity 3.4: Scale numeric features with StandardScaler.
- Activity 3.5: Split the balanced, scaled data into train/test sets (67/33).

Milestone 4: Model Building

- Activity 4.1: Train and evaluate a Decision Tree classifier.
- Activity 4.2: Train and evaluate a Random Forest classifier.
- Activity 4.3: Train and evaluate a K-Nearest Neighbors classifier.
- Activity 4.4: Train and evaluate a tuned XGBoost classifier (n_estimators=150, max_depth=4, learning_rate=0.08).
- Activity 4.5: Compare train/test accuracy and 5-fold cross-validation across all four models; select XGBoost and serialize it (rdf.pkl) along with the fitted scaler (scale1.pkl).

Milestone 5: Application Building

- Activity 5.1: Design home.html, predict.html, and submit.html.
- Activity 5.2: Build the Flask backend (app.py) — routing, form handling, encoding/scaling input the same way as training, and calling the model.
- Activity 5.3: Run, test, and validate the application end-to-end (verified via automated route smoke tests and a 100-request local benchmark).

Milestone 6: Conclusion

Results

Model	Train Accuracy	Test Accuracy	5-fold CV Mean
Decision Tree	100.0%	78.5%	75.0%
Random Forest	100.0%	81.4%	84.1%
KNN	85.8%	74.9%	78.4%
XGBoost (deployed)	96.5%	81.0%	82.1%

XGBoost was selected for deployment: it offers the best balance of generalization (a smaller train/test accuracy gap than Decision Tree or Random Forest) and cross-validated accuracy among the four models tested.

Conclusion

The Smart Lender project demonstrates how machine learning can automate and improve the loan approval process. By analyzing applicant income, employment status, credit history, education, marital status, loan amount, and loan term, the deployed XGBoost model gives credit officers a fast, consistent, data-backed read on applicant risk. The pipeline covers the full lifecycle — data collection, EDA, preprocessing, class balancing, feature scaling, multi-model training, and evaluation — and the Flask web app makes the resulting model usable in a real workflow. Planned next steps include IBM Cloud deployment, real-time credit bureau validation, authentication for different user roles, and containerization for easier hosting.